

QueryGuard AI

Complete Technical Handbook: Questions & Answers, Parameters, Algorithms & Page-by-Page Feature Guide

Document Type: Official System Handbook & Q&A;	Target Audience: New Engineers, DBAs, Architects	Theme: Enterprise Light (#F6F8FB)
Core Engine: HypoPG + PyTorch GNN + SLM	Privacy Guarantee: 99.8% Masked Metadata	Execution Safety: Zero Disk Mutation

Quick Purpose of This Handbook

Welcome to QueryGuard AI! This handbook is written in friendly, plain English so that anyone—whether you are an experienced Database Administrator (DBA), a backend developer, or brand new to database engineering—can clearly understand the three core questions (Parameters, Algorithms, and Optimizations) and navigate every single page of the web dashboard with confidence.

Part 1: The Three Core Technical Questions & Answers

Question 1: What Parameters Are We Using for Our Solution?

In QueryGuard AI, **parameters** are the specific configuration values, thresholds, and measurements that our system watches, tunes, and feeds into our intelligence engine. To make this easy to understand, we group them into four practical categories:

Category	Key Parameters Used	What It Means in Simple Words
1. Database Cost & Hardware Parameters	- seq_page_cost (1.0) - random_page_cost (4.0) - cpu_tuple_cost (0.01) - cpu_operator_cost (0.0025) - work_mem (64MB) - effective_cache_size (4GB)	These are PostgreSQL's internal physics rules. They tell the database how expensive it is to read an un-indexed disk page sequentially (cost 1.0) versus jumping randomly with an index (cost 4.0), and how much CPU effort is needed to process each row.
2. Workload & Telemetry Parameters	- calls (execution count) - mean_exec_time (ms) - p50, p95, p99 latency - shared_blks_read - shared_blks_hit (cache hit ratio) - slow_query_threshold (100ms)	Live heartbeat parameters collected from pg_stat_statements. They tell us which queries are executed millions of times, which queries cause sudden latency spikes (P95/P99), and whether data is found in RAM or dragging from slow disk.
3. Graph Neural Network (GNN) Parameters	- Input vector: 16 dimensions - Hidden layer: 64 dimensions - GNN layers: 2 message-passing - Dropout rate: 0.2 - Learning rate: 0.001 (Adam) - Decision Weights: Cost (0.42), Rows (0.28), Depth (0.18), Cache (0.12)	The settings of our specialized AI model. The 16 input numbers describe each step of a query plan (rows, cost, operator type). The 2-layer GraphSAGE architecture reads the execution tree, and the decision weights mathematically rank what is hurting performance.
4. Privacy & Masking Parameters	- Salted HMAC-SHA256 tokens - Masking target: 99.8% entropy - Table token format: TBL_a1b2 - Column token format: COL_c3d4 - Value masks: [MASKED_INT]	The privacy guardrails. Customer names, passwords, and IDs are replaced with one-way scrambled tokens before any AI or human sees them. Zero customer records or plaintext credentials ever leave the local database.

Question 2: What Algorithms Are We Using?

Instead of using a generic, slow, and expensive cloud AI that hallucinates, QueryGuard AI combines four mathematically rigorous algorithms that run locally in milliseconds:

Algorithm Name	Technical Class	How It Works & Everyday Analogy
1. Graph Neural Network (GraphSAGE / GCN)	Deep Learning on Directed Acyclic Graphs (DAG)	Analogy: An X-ray radiologist reading an entire skeletal scan. A database query plan is not a flat sentence; it is a tree of operations (joins, filters, scans). GraphSAGE aggregates information from child steps up to parent steps, detecting invisible bottlenecks like un-indexed nested loops in sub-4ms.
2. AST Sanitization & Lexical Pseudonymizer	Deterministic Compiler Parsing (SQLGlot)	Analogy: A diplomatic redaction marker. It parses SQL queries into an Abstract Syntax Tree (AST), strips comments and secrets, replaces literals with placeholders, and hashes table/column names into salted HMAC-SHA256 tokens in linear time O(N).
3. System-R Cost Search & Virtual Planner	Dynamic Programming Optimization (PostgreSQL + HypoPG)	Analogy: A GPS recalculating the fastest route using a virtual bridge. HypoPG injects hypothetical index catalog entries into session RAM. PostgreSQL's System-R optimizer re-plans the query dynamically and calculates the exact Cost Delta: $\Delta C = ((\text{Cost_before} - \text{Cost_after}) / \text{Cost_before}) * 100\%$.
4. Deterministic Rule Attribution Engine	Symbolic Multi-Factor Attribution	Analogy: A forensic accountant checking line-by-line expenses. Calculates the exact disk and CPU footprint using formula: $(N_pages * seq_page_cost) + (N_tuples * cpu_tuple_cost)$. It combines GNN weights and database math so DBAs know exactly <i>why</i> a recommendation was made.

Question 3: What Are We Using to Optimize Our Solution?

To make QueryGuard AI ultra-fast, safe, and cost-effective, we optimize the solution across five key technical dimensions:

Optimization Technique	Technology Used	Performance & Safety Advantage
1. Zero-Disk In-Memory Simulation	PostgreSQL HypoPG Extension	Creating a real 10GB index in production takes minutes, locks tables, and consumes expensive disk storage. HypoPG creates <i>hypothetical</i> indexes in session RAM in <15ms with 0 bytes written to disk . We can safely test 100 candidate indexes without risking production downtime.
2. Sub-4ms Local PyTorch GNN Inference	PyTorch C++ Backend + ONNX Ready	Cloud LLMs take 2,000 to 5,000ms per API call and cost per token. Our local GraphSAGE model evaluates query plans in sub-4 milliseconds on standard CPU. It is 500x faster, completely private, and works without an internet connection.
3. Local Air-Gapped SLM Rewriter	Ollama (Llama 3 / Mistral 8B)	When a query needs structural rewriting (e.g., converting an expensive correlated subquery into a clean JOIN), we run a local Small Language Model (SLM) strictly inside the customer's private Docker network. Zero query text ever leaves the company perimeter.
4. Non-Blocking Async Ingestion	FastAPI Async Coroutines + AsyncPG	Telemetry polling from pg_stat_statements runs asynchronously in background worker threads. The database never suffers lock contention or CPU degradation while metrics are gathered.
5. Session-Scoped Virtual Execution	PostgreSQL Transaction Isolation	When the user clicks 'Run Enhanced Query' in the UI, QueryGuard AI attaches the candidate index in an isolated sandbox session, re-runs the query optimizer, and validates that plan cost drops by 90%+ before any DBA approval.

Part 2: Complete Page-by-Page Feature & Working Guide

This section walks through **every single page in the QueryGuard AI web interface**. It is written in simple, step-by-step language so any new team member or DBA can understand what the page does, what each feature and button does, and how it works under the hood.

Global Interface: Top Header & Navigation Sidebar

What it does: The top header and left sidebar stay visible across all pages to give you continuous awareness of database health, security status, and fast one-click navigation.

Features & How they work:

- **Connection Status Badge:** Displays a pale green pill saying Connected (PG 16). It pings the backend every few seconds to confirm the target database is healthy and responding.
- **Environment Indicator:** Displays Local Sandbox (pale amber), guaranteeing that all simulations run in an isolated test environment without touching live production data.
- **Privacy Verification Pill:** Displays Privacy Protected (99.8%) in pale teal. Shows that the SQL sanitization gateway is actively stripping personal data and identifiers.
- **Quick Analyze Query Button:** A bright teal button in the header that instantly takes you to the query inspection workspace from anywhere in the app.
- **Sidebar Navigation:** Clean white sidebar with light gray hover states (#F1F5F9) and distinct dark icons for all 10 core pages.

Page 1: Overview Dashboard (Route: /)

What this page does: The Overview Dashboard is the *executive mission control* for your database. It translates thousands of complex internal database statistics into four high-level health cards, visual charts, and actionable alerts so you can understand your database status in 5 seconds.

Key Features & How they work:

1. **Workload Health Radial Score:** A circular score gauge (0-100%). A score of 90%+ means your database is running smoothly with high buffer cache hits and minimal slow queries.
2. **Bottleneck Distribution Donut Chart:** Replaces raw text with a clean color-coded pie chart showing the proportion of database issues (Sequential Scans in Red, Nested Loops in Amber, Hash Spills in Blue, Lock Waits in Purple).
3. **Latency Spectrum Chart:** Plots average latency (Teal line) against 95th-percentile tail latency (Red line) over time, alerting you to sudden traffic spikes.
4. **Top Priority Action Card:** Automatically surfaces the single worst query dragging down database performance, featuring a red warning accent and a direct Review Bottleneck button to jump straight to the fix.

Page 2: Live Workload Monitor (Route: /workload)

What this page does: Shows what is happening inside the database *right now*. Instead of manually running terminal commands like top or querying system views, this page streams real-time execution statistics.

Key Features & How they work:

1. **Throughput Gauge (QPS):** Shows total Queries Per Second currently being handled by the engine.
2. **Live Execution Latency Bar & Line Chart:** Displays millisecond execution times across the active query stream.
3. **Identified Problems Donut Chart:** Aggregates the root causes of recent slow queries into categorized percentages.
4. **Live Query Telemetry Table:** A live-updating table listing query fingerprints, execution counts (calls), total time, and cache hit ratios. Clicking any row opens its detailed execution profile.

Page 3: Analyze Query Workspace (Route: /analyze)

What this page does: This is the heart and laboratory of QueryGuard AI. You paste any slow or problematic SQL query here, and the system scrubs its private data, visually diagnoses its bottlenecks, runs an in-memory virtual simulation, and lets you test an enhanced version immediately.

Step-by-Step Features & How they work:

1. **SQL Editor & Sample Loader:** A clean light code editor. You can type your own query or click Load Sample Slow Query to test an un-indexed multi-table JOIN.
2. **Privacy Sanitization Gateway:** The moment you submit a query, the SQLGlot AST engine replaces customer IDs, numbers, and strings with tokens. A green audit badge shows the 99.8% privacy guarantee.
3. **Interactive Plan Graph:** Displays the visual execution tree. The root bottleneck is highlighted in **pale red**, secondary bottlenecks in **pale amber**, and optimized steps in **pale green**.
4. **Enhanced Query & Virtual Execution:** When the AI suggests an index, you can click Run Enhanced Query. QueryGuard AI creates the index virtually in session RAM using HypoPG, re-runs the optimizer, and displays a verified cost drop of up to **93.3% with 0 disk writes!**
5. **Explainability (Rule + GNN) Tab:** Shows the exact mathematical cost formula $((N_pages * seq_cost) + (N_tuples * cpu_cost))$, the 2-layer GraphSAGE topological proof, an operator impact chart, and the exact GNN feature weights.

Page 4: Slow Queries Catalog (Route: /slow-queries)

What this page does: Automatically tracks and ranks every query whose execution time exceeds the safety threshold ($\geq 100ms$). It acts as an automated triage center for database slowdowns.

Key Features & How they work:

1. **P95 & P99 Latency Ranking:** Instead of just sorting by average time, it highlights queries that cause random severe spikes for users.
2. **Frequency vs. Duration Multiplier:** A query that takes 200ms but runs 1,000,000 times a day is far more damaging than a 5-second report that runs once a week. This table ranks queries by cumulative database impact.
3. **One-Click Analysis:** Clicking the Analyze button next to any slow query instantly loads it into the Analyze Query workspace.

Page 5: Recommendations & Approvals (Route: /recommendations)

What this page does: Where AI intelligence meets human-in-the-loop safety. QueryGuard AI never alters production database schemas automatically. Instead, it prepares clear, verified recommendations for DBA approval.

Key Features & How they work:

1. **Recommendation Cards:** Displays candidate indexes (e.g., `CREATE INDEX idx_orders_customer_id ON orders(customer_id)`) with estimated cost reduction (e.g., -87.4%) and storage estimate.
2. **DBA Approval Drawer:** Clicking Review & Approve opens a sliding drawer with full risk analysis, benefit score, and side-by-side plan comparison.
3. **Automated Rollback DDL:** Every approved recommendation includes an automatically generated rollback command (e.g., `DROP INDEX CONCURRENTLY idx_...`) so changes can be reverted instantly if needed.

Page 6: What-If Simulations Showcase (Route: /simulations)

What this page does: A dedicated flight-simulator for database administrators. It allows DBAs to experiment with hypothetical indexes, composite keys, and table reorganizations without touching real disks.

Key Features & How they work:

1. **Hypothetical Index Workbench:** Pick a table, pick one or more columns, and test index creation in RAM.
2. **Side-by-Side Tree Comparison:** Shows the original execution plan on the left (Red Seq Scan) and the hypothetical plan on the right (Green Index Scan).
3. **Resource & Storage Estimator:** Calculates how many megabytes of RAM and disk the index would consume if made permanent.

Page 7: Model Insights & AI Explainability (Route: /model-insights)

What this page does: Provides total transparency into the AI models powering QueryGuard AI. It proves that the system is based on verified scientific machine learning, not black-box guesswork.

Key Features & How they work:

- 1. **GNN Architecture Map:** Visualizes the 2-layer GraphSAGE neural network, showing how 16-dimensional node embeddings flow through 64-dimensional hidden representations.
- 2. **50-Epoch Training Curve:** Plots the loss decay and accuracy convergence graph over 50 training epochs, reaching 94.6% validation accuracy.
- 3. **Confusion Matrix Heatmap:** A color-coded grid proving the model's precision in classifying Sequential Scans, Hash Joins, Nested Loops, and Materialize operators without false positives.
- 4. **Dataset Topology Donut:** Shows the 80/10/10 split across training, validation, and test workloads.

Page 8: Audit Trail & Compliance Ledger (Route: /audit)

What this page does: An immutable compliance record of every action taken in QueryGuard AI. It provides enterprise security teams with forensic evidence of database safety.

Key Features & How they work:

- 1. **Cryptographic Event Ledger:** Every query analysis, simulation run, approval, and rejection is stamped with an immutable timestamp, user ID, and action hash.
- 2. **Zero-Mutation Verification:** Proves cryptographically that during analysis, zero DDL or DML writes occurred against the live production tables.
- 3. **Filterable Timeline:** Search events by actor, risk level, or approval status for SOC2 and ISO-27001 compliance reviews.

Page 9: Benchmark Datasets (Route: /benchmarks)

What this page does: Allows testing and benchmarking against industry-standard workloads so you can evaluate database tuning in a controlled laboratory environment.

Key Features & How they work:

- 1. **Standard TPC-H Workload:** Includes industry-standard analytical queries (scale factor 0.1 and 1.0) with multi-table joins and complex aggregations.
- 2. **Synthetic E-Commerce Workload:** Simulates high-velocity online shopping carts, orders, customer records, and inventory lookups.
- 3. **One-Click Sandbox Seeding:** Easily reset or reload benchmark tables with a single click to run repeatable performance comparisons.

Page 10: Settings & Connection Management (Route: /settings)

What this page does: Controls the target database connections, privacy masking sensitivity, and telemetry collection frequencies.

Key Features & How they work:

- 1. **Target Database Credentials:** Secure configuration of PostgreSQL connection URI with read-only permission enforcement.
- 2. **Privacy Self-Test Tool:** Paste sample proprietary text to verify that names, credit cards, and table names are scrambled into HMAC tokens before connecting.
- 3. **Telemetry Cadence Slider:** Set the frequency of pg_stat_statements polling (e.g., every 5s, 15s, or 60s) to balance freshness with low overhead.

Summary: 30-Second Quick Reference Cheat Sheet

If You Want To...	Go To This Page	Key Feature To Use
-------------------	-----------------	--------------------

Check if database is healthy right now	Overview (/)	Workload Health Score + Bottleneck Donut
See live incoming queries & traffic spikes	Live Workload (/workload)	Live QPS Gauge + Latency Line Chart
Paste a slow query, test virtual index & run fix	Analyze Query (/analyze)	SQL Editor -> Run Enhanced Query (HypoPG)
Understand the math and AI behind the diagnosis	Analyze Query (/analyze)	Explainability (Rule + GNN) Attribution Tab
Find the worst queries running in the database	Slow Queries (/slow-queries)	P95/P99 Latency Filter & Impact Rank
Approve or reject a suggested index safely	Recommendations (/recommendations)	DBA Approval Drawer with Rollback DDL
Test what-if index ideas without writing to disk	Simulations (/simulations)	HypoPG In-Memory Plan Comparator
Audit security, actions and zero-mutation proofs	Audit Trail (/audit)	Cryptographic Event Ledger & Export Log

Key Takeaway for New Engineers

QueryGuard AI Rule of Thumb: No matter how complex a database issue appears, you can always diagnose it safely in three clicks: **(1)** Review on the Overview dashboard, **(2)** Analyze & Run Enhanced in the sandbox, and **(3)** Review the Rollback DDL in Recommendations before applying to production.